Statistical and Machine Learning

Tree based methods and boosting

Katarzyna Reluga

Tree based methods and boosting

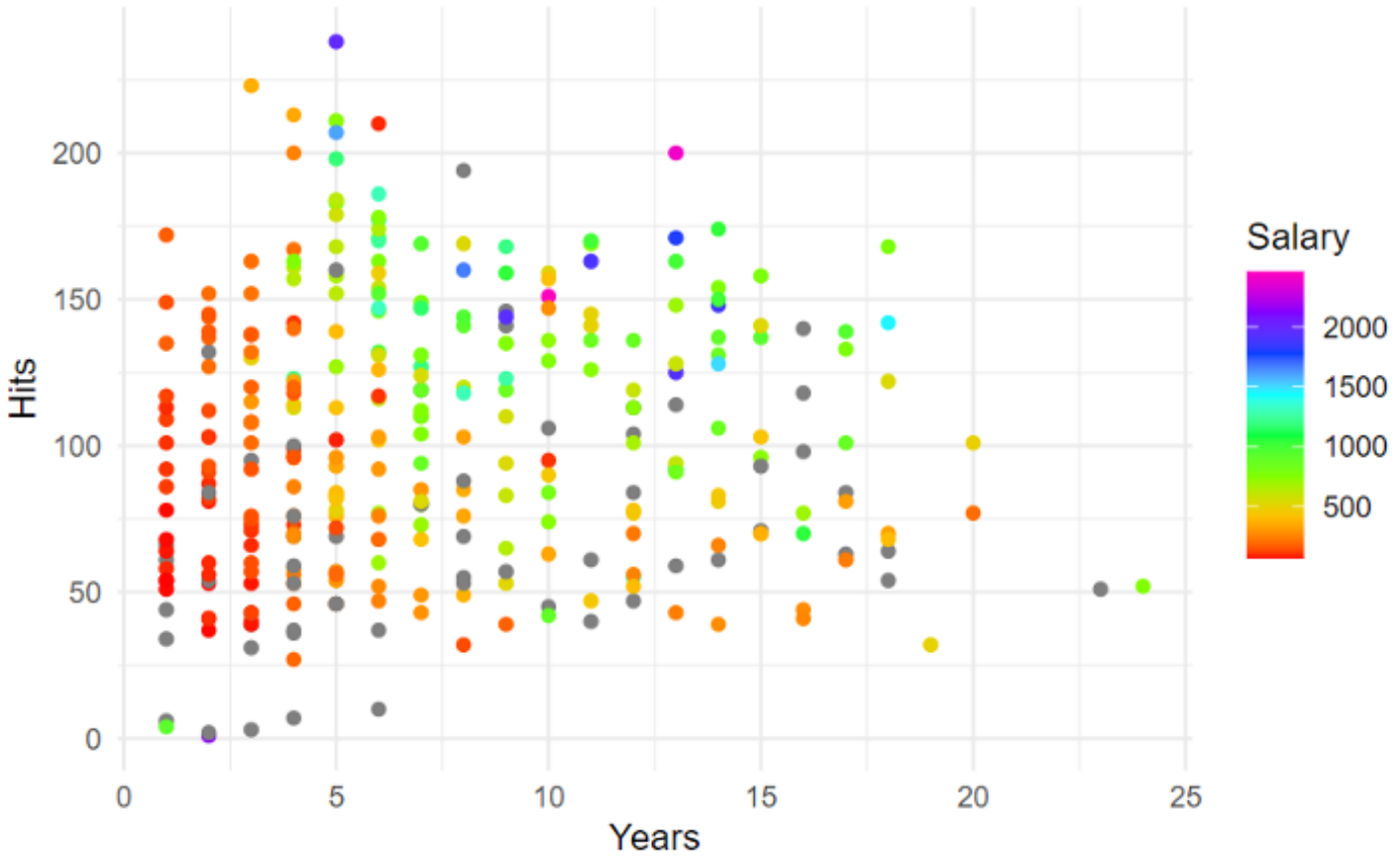
- Decision trees
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - XGBoost

Tree based methods and boosting

- **Decision tree**
  - **Introduction**
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - XGBoost

Decision tree

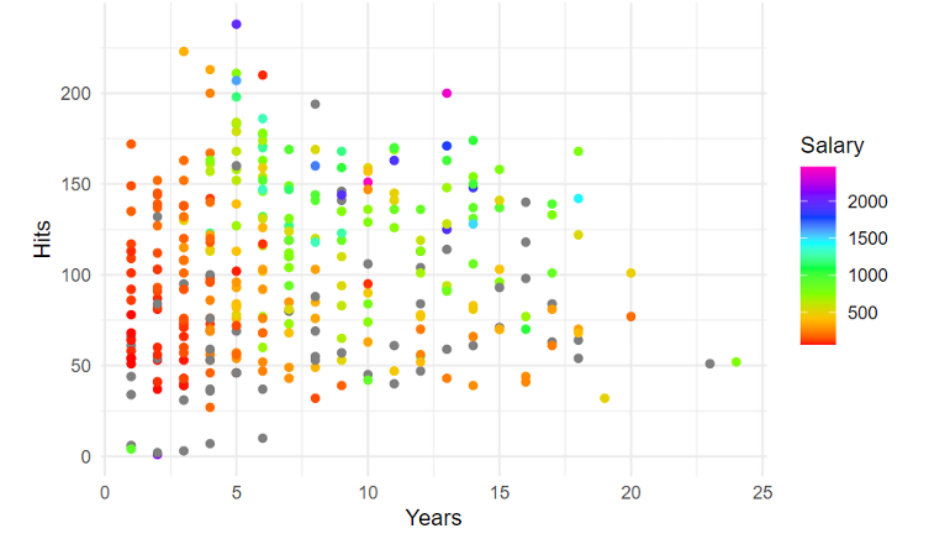
- Rule based classification (if-else)
- Warm-up: how would you stratify baseball players' salary data?
  - Salary is color-coded (NA values coded gray)

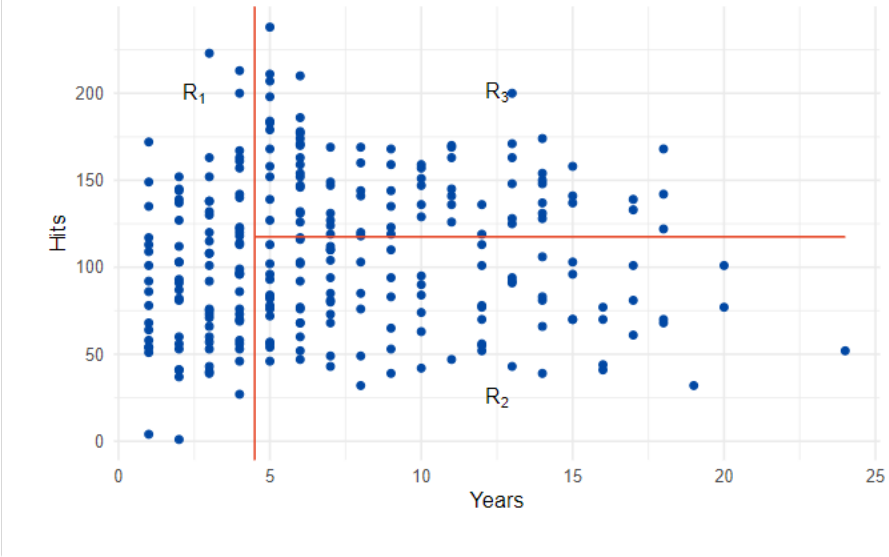


Decision tree

- How would you stratify baseball players' salary data?

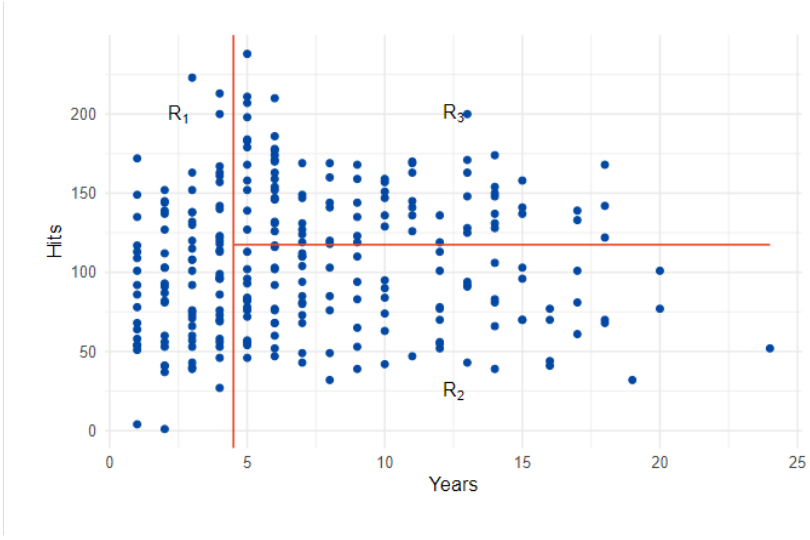
$$R_1 = \{X | Years < 4.5\}$$
$$R_2 = \{X | Years \geq 4.5, Hits < 117.5\}$$
$$R_3 = \{X | Years \geq 4.5, Hits \geq 117.5\}$$

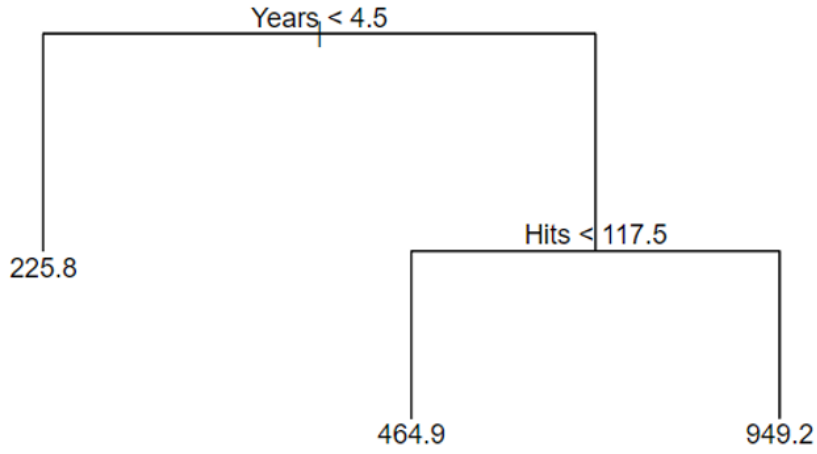




Decision tree

$$R_1 = \{X | Years < 4.5\}$$
$$R_2 = \{X | Years \geq 4.5, Hits < 117.5\}$$
$$R_3 = \{X | Years \geq 4.5, Hits \geq 117.5\}$$

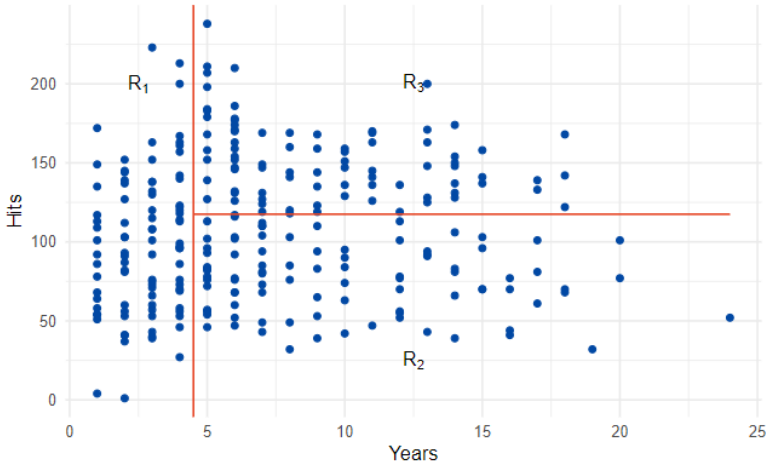


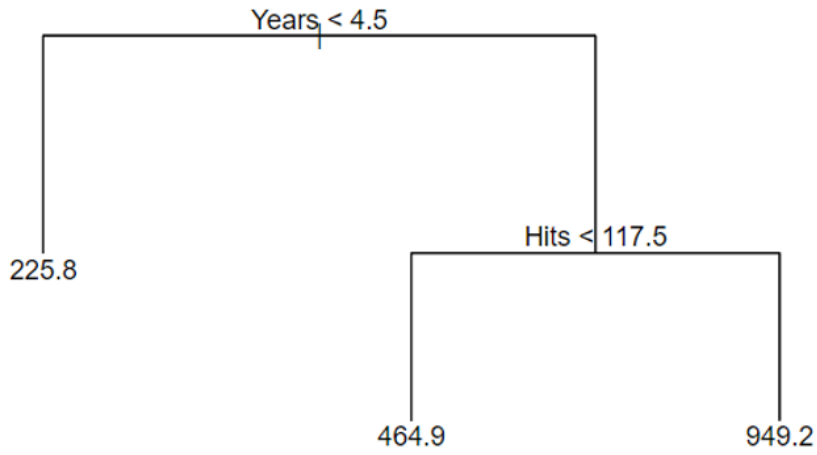


- **If** Years < 4.5, then the salary will be less than XXX.
- **Else if** Hits < 117.5, then the salary will be around XXX.
- **Otherwise**, the salary will be XXX.

Terminology for Decision Trees

- Decision trees are often drawn **upside down**: the **root node** at the top, the **branches** move downward, the **leaves** at the bottom.
- **Internal nodes** are points where the predictor space is split, for example $X_j < c$.
- **Leaves**, also called **terminal nodes**, are nodes where the tree stops splitting. Each leaf corresponds to a final region of the predictor space, e.g. R_1, R_2, R_3
- Predictions are made by summarising the training observations:
  - regression tree: average response in the leaf,
  - classification tree: majority class/class probabilities in the leaf.





Questions worth answering

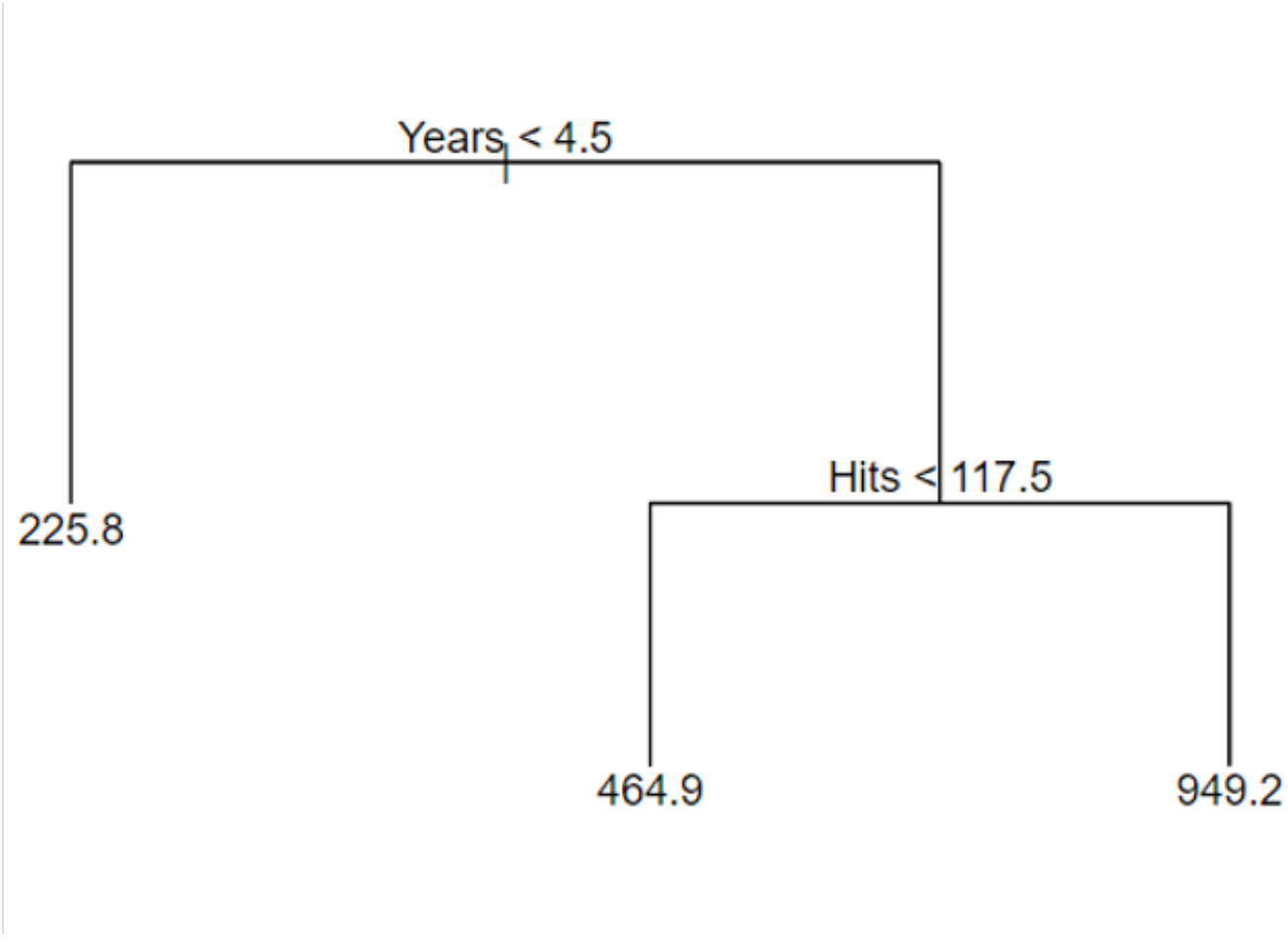
- Q1: What is the response in each partitioned region? ($\hat{f}(x) =?$ if $x \in R_m?$)
- Q2: Which variables should we use to partition the sample space? (e.g. Years or hits?)
- Q3: How do we split the selected variable? (e.g. $\mathit{Years} < 4.5$ or $\mathit{Years} < 1$)
- Q4: When do we stop splitting?

Trees, forest, bagging and boosting

- Decision trees
  - Introduction
  - **Classification and Regression Tree (CART)**
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - XGBoost

Classification And Regression Tree (CART)

- Introduced by Breiman (1984)
- Binary splits are constructed top-down
- Constant prediction in each leaf



Q1: What is the response in each region?

Once a region R_m has been formed, the prediction is constant inside that region.

Regression trees $\rightarrow$ **mean response**

- Prediction: sample mean $\hat{c}_m = \frac{1}{|R_m|} \sum_{i:x_i \in R_m} y_i$
- $\hat{f}(x) = \hat{c}_m$ if $x \in R_m$.
- Example, if $R_1 = \{\text{Years} < 4.5\}$, the average salary $c_m = 120$ then $\hat{f}(x) = 120$ for all $x \in R_1$.

Classification tree $\rightarrow$ **majority class** or **class probabilities**.

- Compute class probabilities (proportions) in a terminal region R_m: $\hat{\pi}_{mk} = \frac{1}{|R_m|} \sum_{i:x_i \in R_m} I(y_i = k)$.
- Prediction: the majority class: $\hat{f}(x) = \arg\max_k \hat{\pi}_{mk}$, $x \in R_m$.
- Example: if in region R_m we have $\hat{\pi}_{m,0} = 0.2$, $\hat{\pi}_{m,1} = 0.8$, then $\hat{f}(x) = 1$ for all $x \in R_m$.

Q2: Which variable should we use to partition the sample space?

- At a given node, CART considers all predictors $X_1, \ldots, X_p$.
- For each predictor X_j, it considers possible split points s, giving two regions: $R_1(j, s) = \{X : X_j < s\}$, $R_2(j, s) = \{X : X_j \geq s\}$
- For example, if the predictors are `Years` and `Hits`, CART compares splits such as Years < 4.5 and Hits < 117.5.
- The selected variable is the one that gives the largest reduction in prediction error (defined later).

Q3: How do we choose the split points?

For a continuous predictor X_j, CART does **not** choose split points randomly.

At a given node, it sorts the distinct observed values of X_j:

$$x_{(1)j} < x_{(2)j} < \cdots < x_{(K)j}.$$

The candidate split points are usually the midpoints between consecutive distinct values:

$$s_k = \frac{x_{(k)j} + x_{(k+1)j}}{2} \qquad k = 1, \ldots, K-1.$$

For example, if the observed values of `Years` are 1, 2, 3, 6, 9, then the candidate splits are 1.5, 2.5, 4.5, 7.5.

CART evaluates each candidate split and chooses the one that gives the smallest prediction error.

For each predictor, CART checks all candidate midpoints between consecutive distinct observed values.

Q3: Measuring the prediction error to choose the splits

For a candidate split (j, s),

$$R_1(j,s) = \{x : x_j < s\}, \qquad R_2(j,s) = \{x : x_j \geq s\}$$

CART chooses the split that gives the best improvement in prediction error.

Regression

- Error measured through RSS

$$RSS(j,s) = \sum_{i:x_{ij}<s} (y_i - \hat{c}_1)^2 + \sum_{i:x_{ij}\geq s} (y_i - \hat{c}_2)^2$$

$$\hat{c}_1 = \frac{1}{|R_1|} \sum_{i:x_i \in R_1} y_i, \qquad \hat{c}_2 = \frac{1}{|R_2|} \sum_{i:x_i \in R_2} y_i$$

.

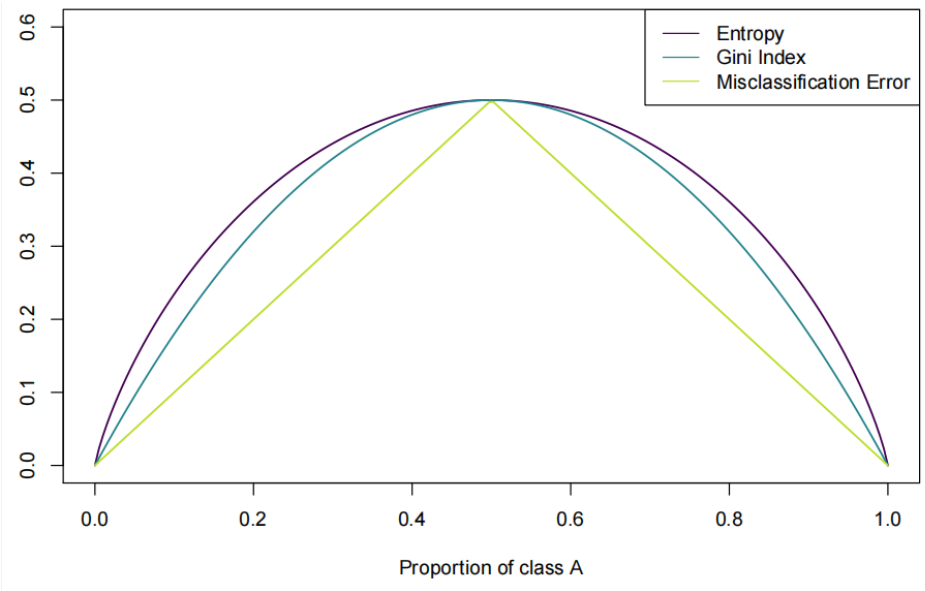
Q3: Measuring the prediction error to choose the splits

Classification (K classes)

- CART chooses the split (j, s) that makes the child nodes as **pure** as possible.
- Let $\hat{\pi}_{mk} = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} I(y_i = k)$.
- Error measured through **impurity** measurements
  - Misclassification error $i_1(R_m) = 1 - \max_k \hat{\pi}_{mk}$
  - Gini index: $i_2(R_m) = \sum_{k=1}^{K} \hat{\pi}_{mk}(1 - \hat{\pi}_{mk}) = 1 - \sum_{k=1}^{K} \hat{\pi}_{mk}^2$
  - (Shannon) entropy: $i_3(R_m) = -\sum_{k=1}^{K} \hat{\pi}_{mk} \log(\hat{\pi}_{mk})$
- CART chooses the split (j, s) minimising the weighted impurity: $i_w(j, s) = \frac{|R_1|}{|R|} i(R_1) + \frac{|R_2|}{|R|} i(R_2)$.

CART – impurity measurements

- For K=2



$$i_1(\pi_1) = \begin{cases} \pi_1, & \pi_1 \leq \frac{1}{2} \\ 1 - \pi_1, & \pi_1 > \frac{1}{2} \end{cases}$$

$$i_2(\pi_1) = 1 - \pi_1^2$$

$$i_3(\pi_1) = -\pi_1 \log(\pi_1) - (1 - \pi_1) \log(1 - \pi_1)$$

Q3: Does CART compare all possible splits?

CART may compare many possible splits.

However, this is still manageable because:

- the data are sorted once for each predictor,
- candidate RSS values can be updated efficiently while moving through the sorted values,
- only splits at observed-data boundaries need to be checked.

Thus CART does not search over all real numbers s. It searches over a finite set of data-determined candidates.

Q3: Efficient computation of split quality – regression example

After sorting X_j, CART moves the split point from left to right.

At each candidate split, observations move from the right region to the left region.

The region means and RSS can be updated using running sums:

$$\sum_{i \in R_1} y_i, \qquad \sum_{i \in R_1} y_i^2, \qquad \sum_{i \in R_2} y_i, \qquad \sum_{i \in R_2} y_i^2.$$

For a region R, the RSS can be computed as

$$\sum_{i \in R} (y_i - \bar{\hat{c}}_R)^2 = \sum_{i \in R} y_i^2 - \frac{\left(\sum_{i \in R} y_i\right)^2}{|R|}.$$

So CART can evaluate many candidate splits efficiently, without refitting a model from scratch for every split.

Q4: When do we stop splitting?

A tree can keep splitting until each terminal node contains very few observations.

This usually gives a very flexible tree with low training error, but high variance.

Common stopping rules include:

- minimum number of observations in a terminal node,
- maximum tree depth,
- minimum improvement in RSS/impurity after a split.

For example, we may stop splitting if a node contains fewer than 10 observations, or if the reduction in RSS is too small.

Q4: Cost-complexity pruning

Instead of stopping early, CART often first grows a large tree and then prunes it.

For a tree T, define the cost-complexity criterion

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} L(y_i, \hat{f}_m) + \alpha |T|$$

where:

- the first term measures training error,
- the second – penalizes the size of the tree, with $|T| =$ number of terminal nodes.

The tuning parameter $\alpha \geq 0$ controls complexity:

- $\alpha = 0 \Rightarrow$ large tree
- α large $\Rightarrow$ smaller tree.

Usually, α is chosen by cross-validation. The parameter α penalizes tree complexity, so it acts as a regularization parameter.

Q4: Loss functions in cost-complexity pruning

Regression trees

RSS loss:

$$L(y_i, \hat{f}_m) = (y_i - \hat{f}_m)^2, \quad \hat{f}_m = \hat{c}_m = \frac{1}{|R_m|} \sum_{i:x_i \in R_m} y_i$$

Classification tress

Misclassification loss

$$L(y_i, \hat{f}_m) = I\{y_i \neq \hat{f}_m\}, \quad \hat{f}_m = \arg\max_k \hat{\pi}_{mk}$$

where $\hat{f}_m$ is the predicted class in terminal node R_m and

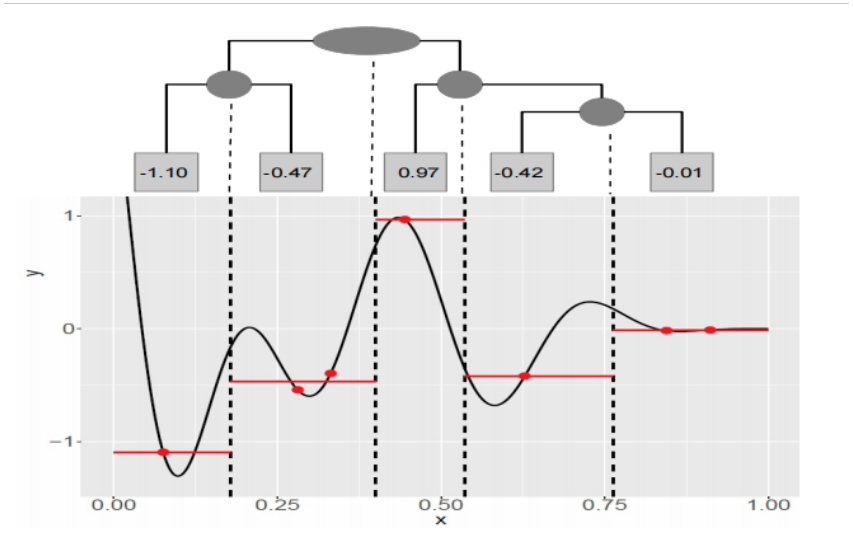
$$\frac{1}{|R_m|} \sum_{i:x_i \in R_m} I\{y_i \neq \hat{f}_m\} = 1 - \max_k \hat{\pi}_{mk} = i_1(R_m)$$

where $i_1(\cdot)$ a misclassification error.

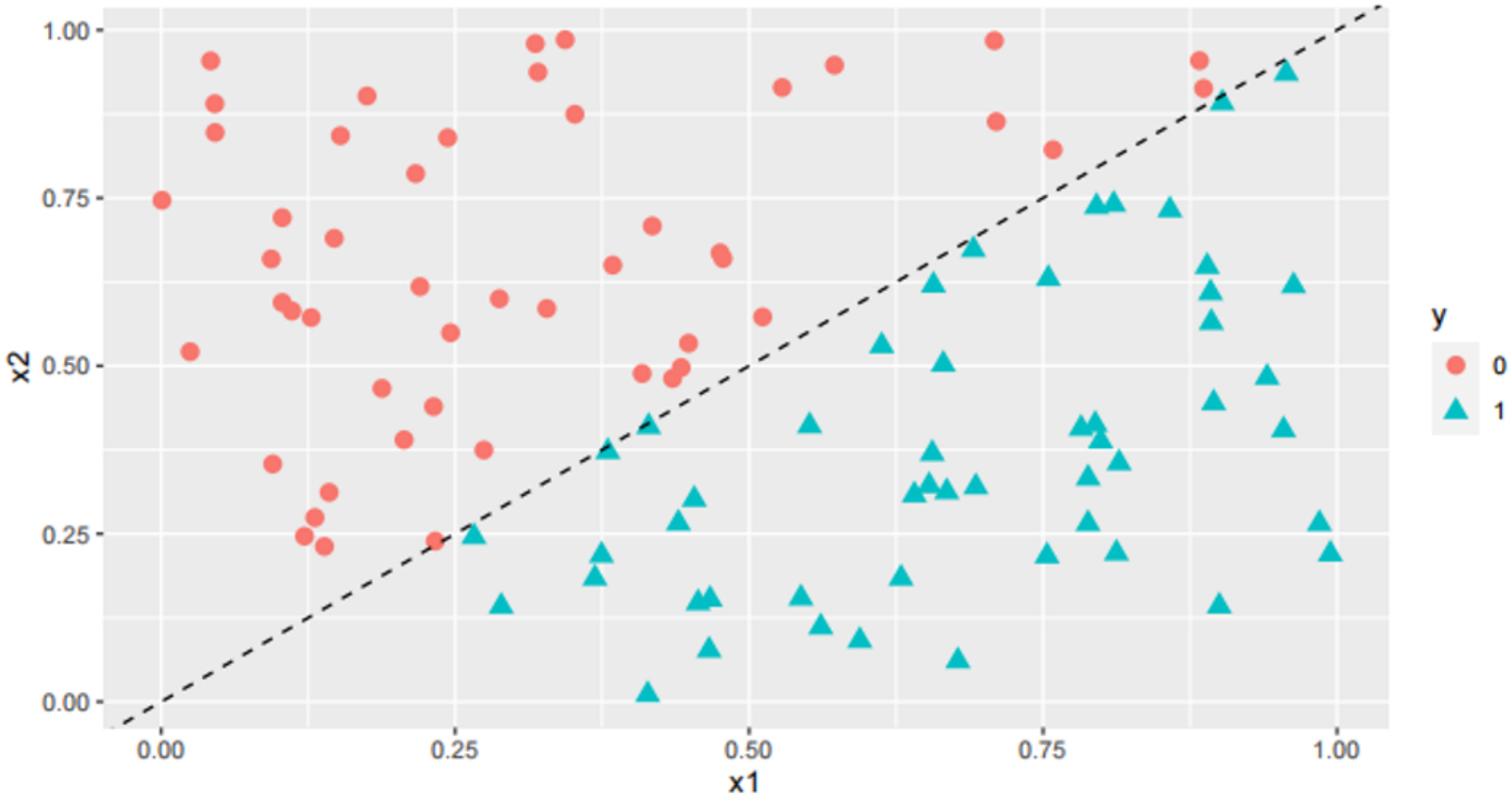
CART – regression example

$$y = \sin(4x - 4)(2x - 2)^2 \sin(20x - 4)$$

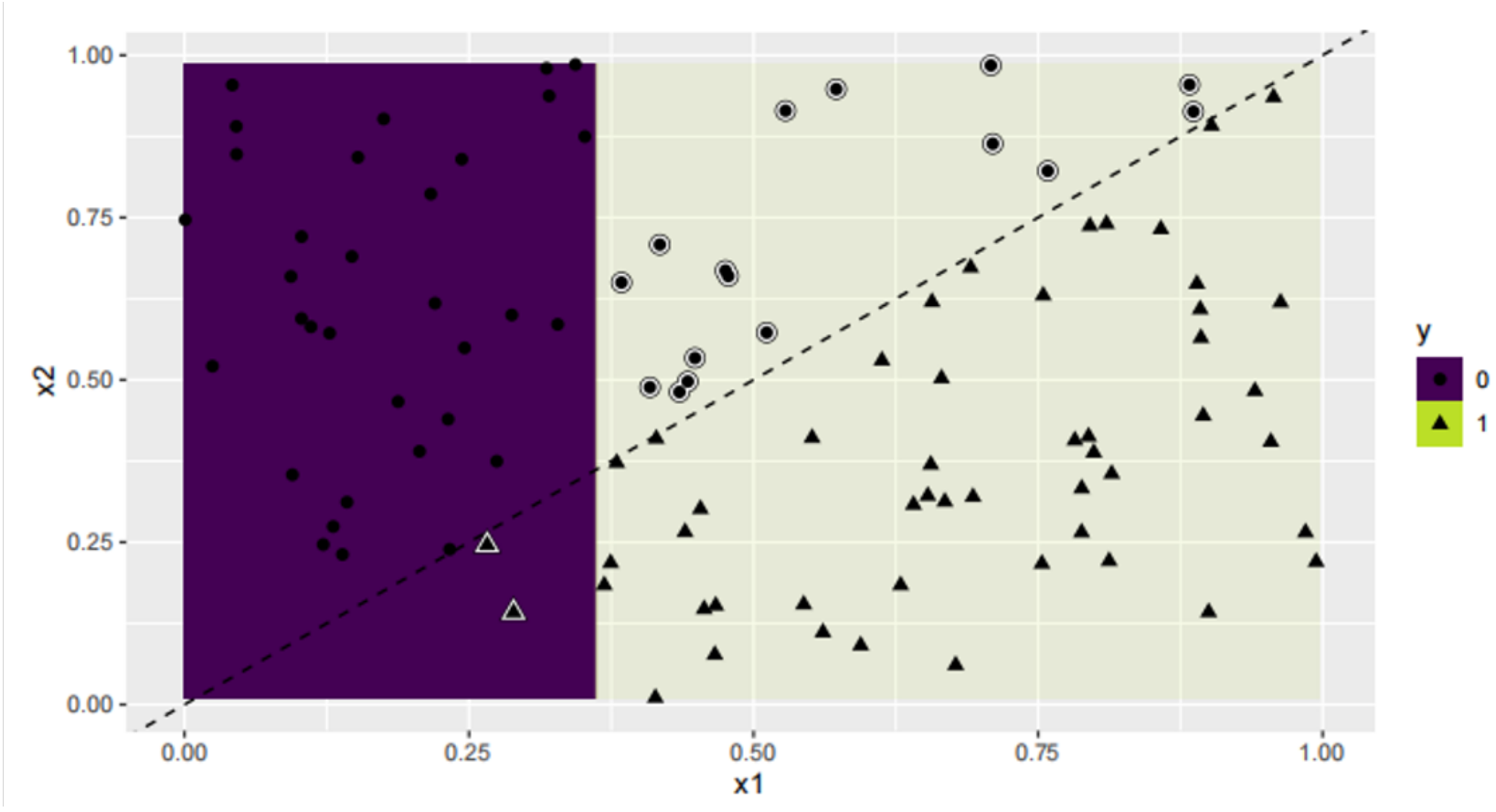
x	y
0.075	-1.095
0.281	-0.543
0.331	-0.396
0.445	0.965
0.628	-0.421
0.845	-0.018
0.912	-0.011



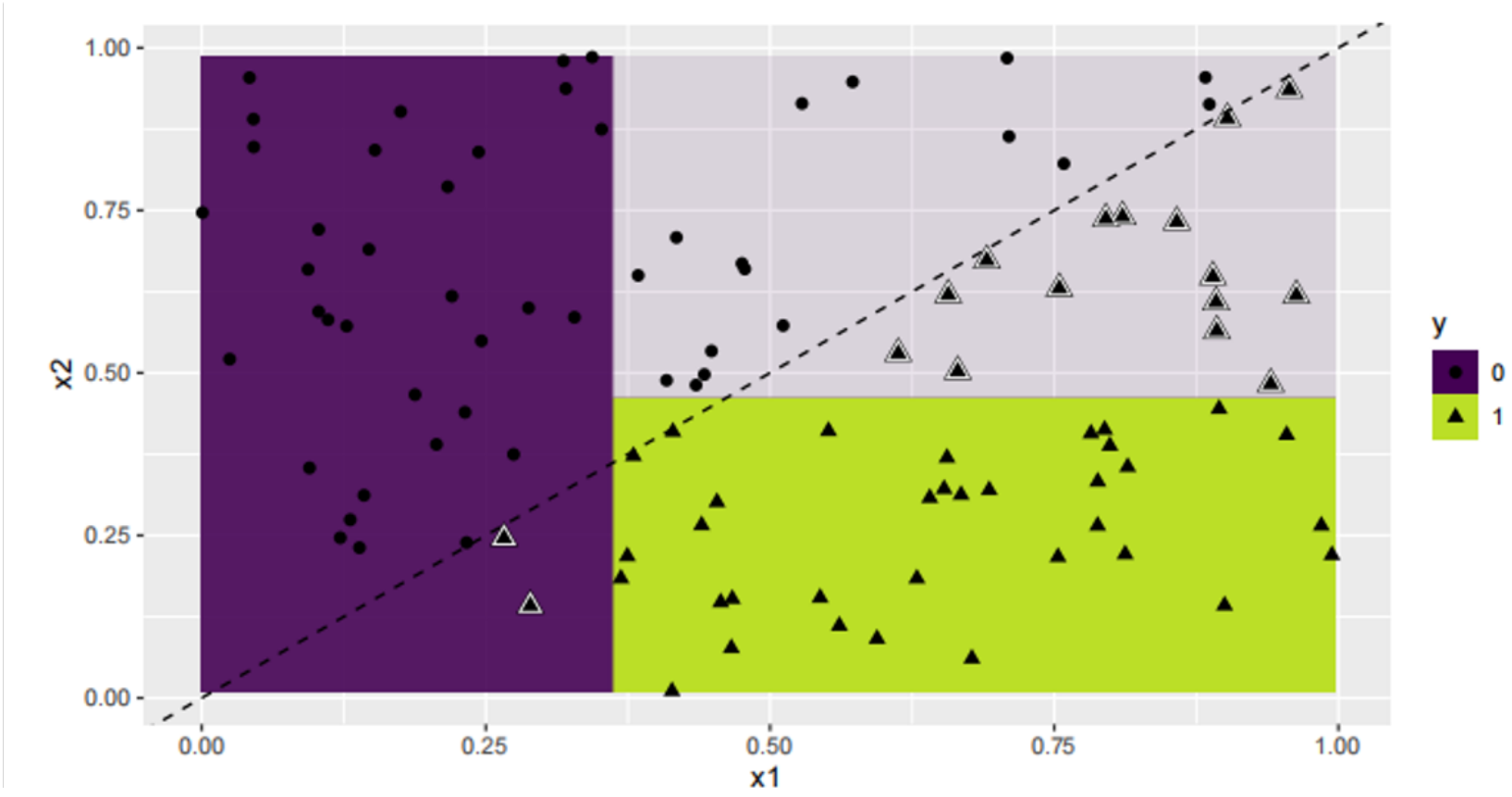
CART vs. linear classifier



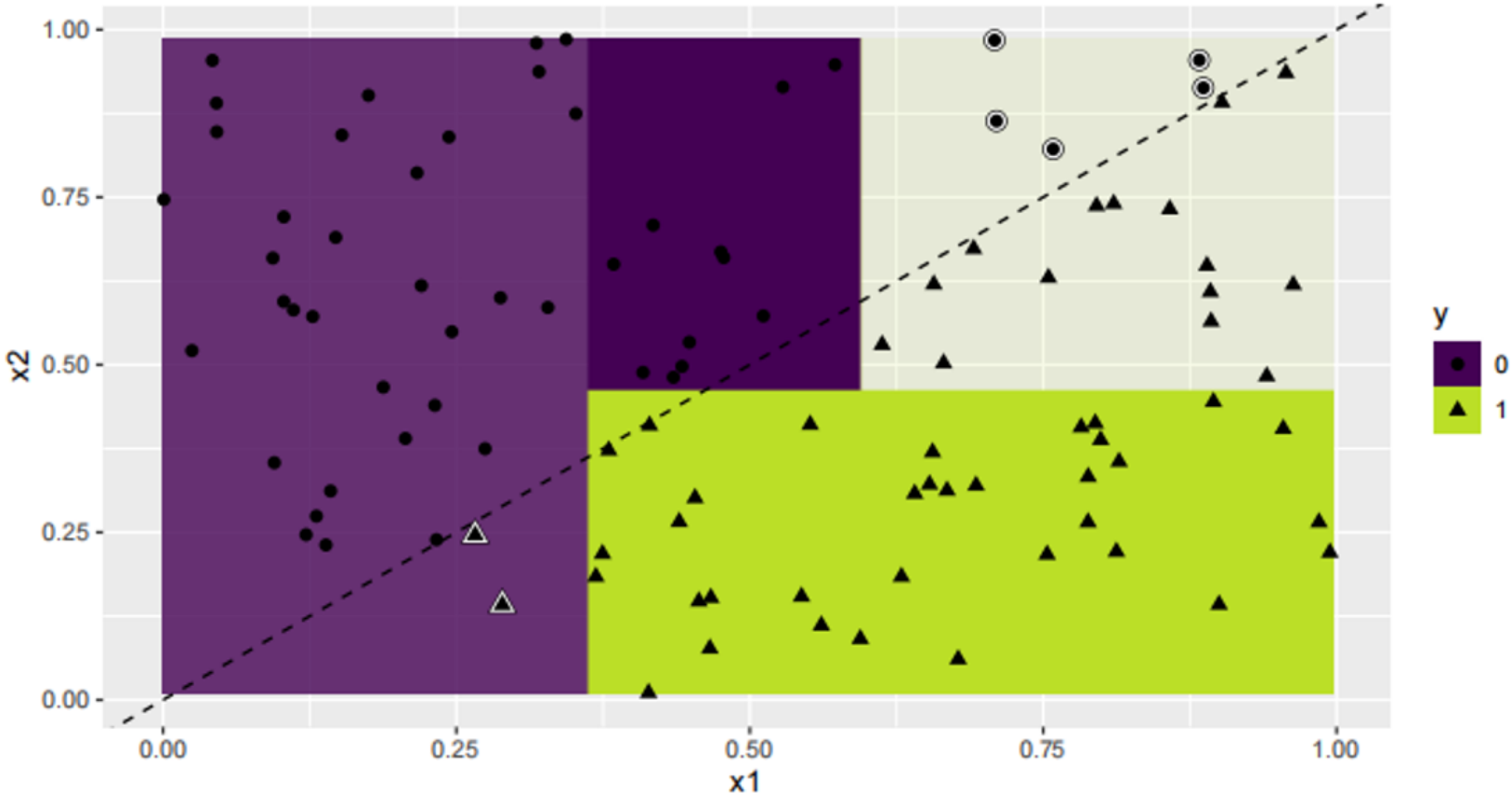
CART vs. linear classifier



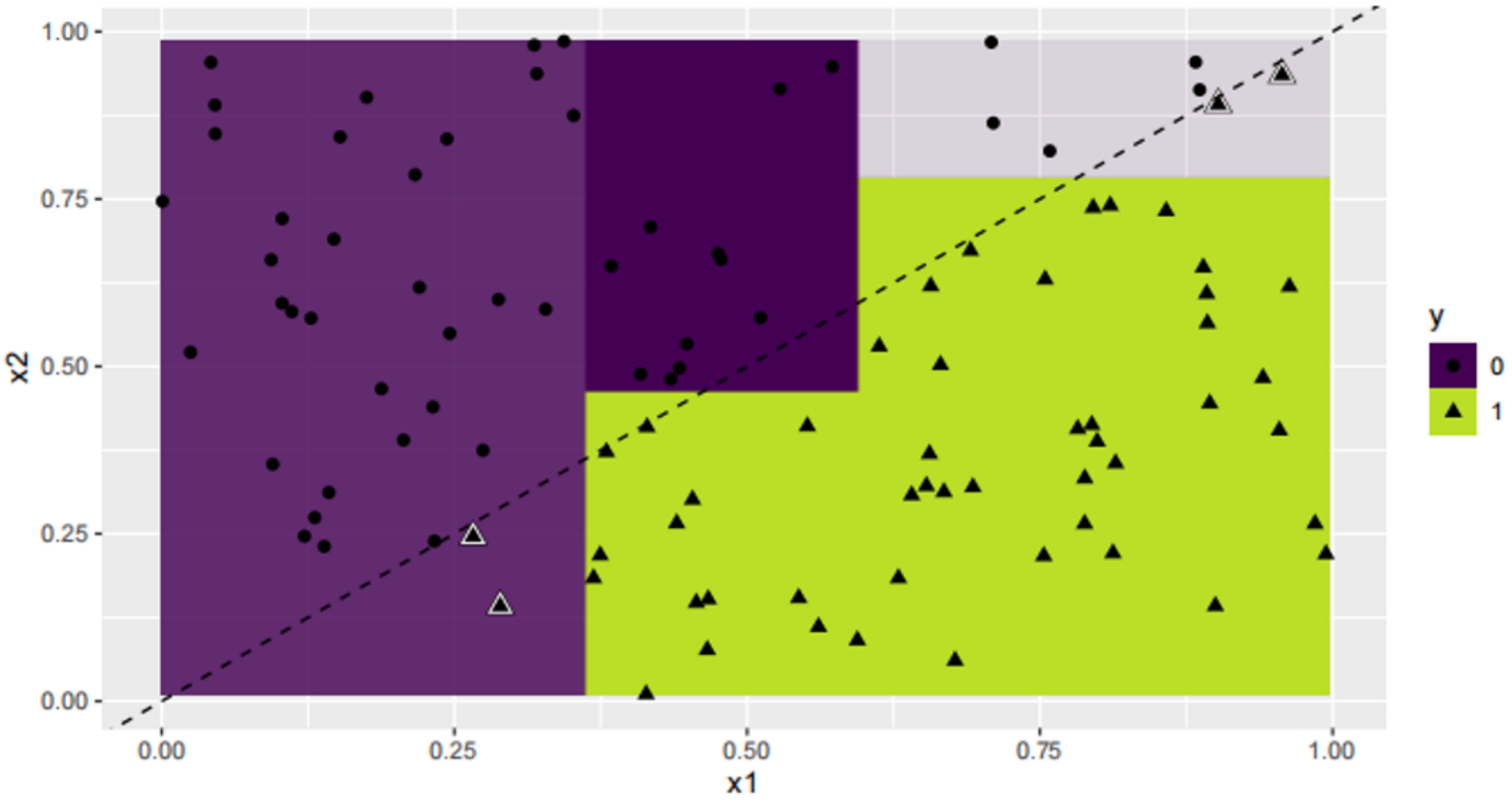
CART vs. linear classifier



CART vs. linear classifier



CART vs. linear classifier



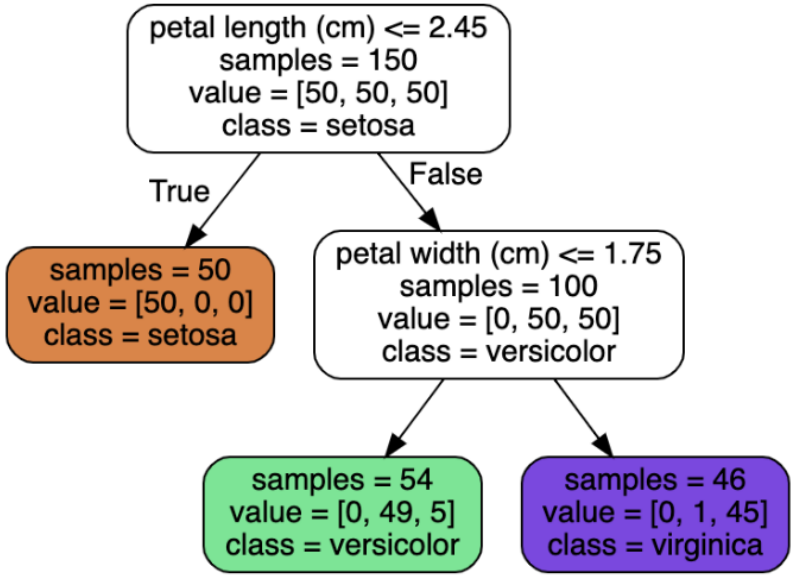
CART pros

- **They are relatively easy to interpret.**
- They can easily handle mixed discrete and continuous inputs.
- They are insensitive to monotone transformations of the inputs (because the split points are based on ranking the data points), so there is no need to standardize the data.
- They perform automatic variable selection.
- They are relatively robust to outliers.
- They are fast to fit, and scale well to large data sets.
- They can handle missing input features.

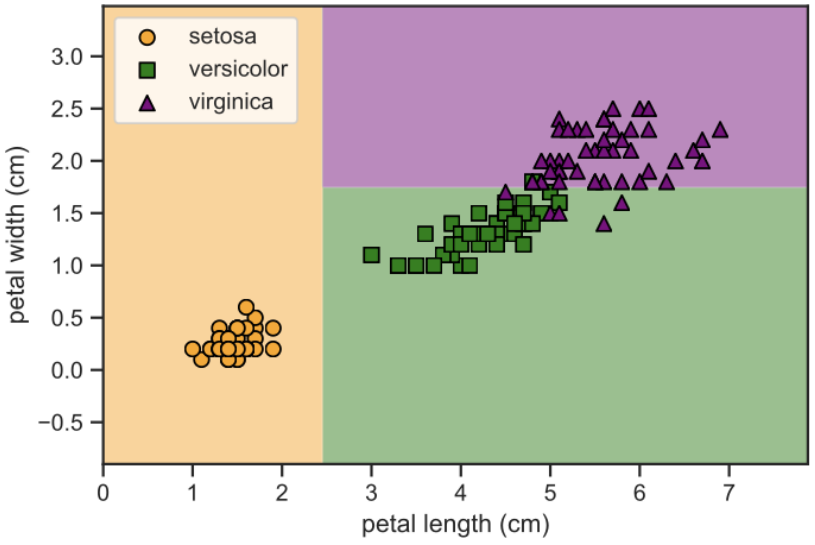
CART cons

- Trees do not have the same level of predictive accuracy as some of the other regression and classification approaches we have discussed.
- Trees are unstable: small changes to the input data can have large effects on the structure of the tree.
- A single decision tree is prone to over-fitting.
- Complex tree also very hard to interpret!

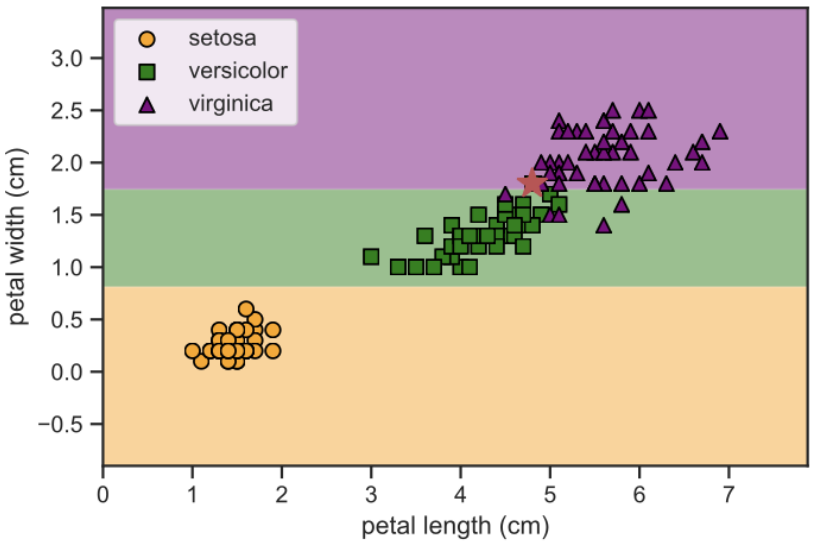
Unstability of decision tree



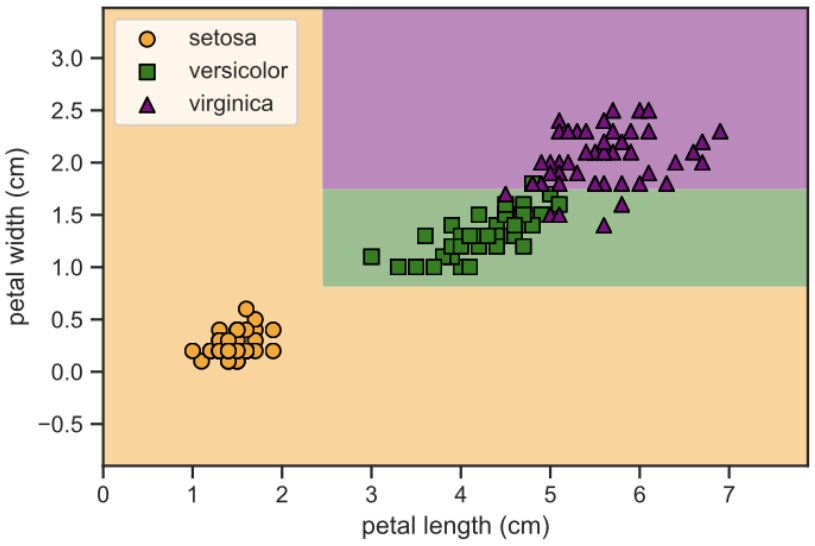
(a)



(b)



(c)



(d)

Tree based methods and boosting

- Decision tree
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - XGBoost

Ensemble methods/learners

- **Problem**: decision trees (or another base learner) have high variance
- **Need**: to reduce the variance
- **Idea**: use more trees and aggregate the results by taking:
  - an average over base learners $\rightarrow$ regression problems;
  - a majority vote of base learners $\rightarrow$ classification problems.

Bagging (Bootstrap Aggregating)

Bootstrap steps

- Input: set D containing m training observations
- Create bootstrap sample $b^{[i]}$ by drawing m observations at random with replacement from D
- Remarks:
  - Since the sampling is done with replacement,some observations appear more than once, others do not appear at all
  - The probability of not being selected in 1 draw is: $1 - \frac{1}{m}$ and in any of the m draws: $\left(1 - \frac{1}{m}\right)^m \approx e^{-1} \approx 0.37$
  - Each bootstrap sample leaves out (**out-of-bag observations**) about $0.37|D|$ observations on average.

Aggregating steps

- Create k bootstrap samples $b^{[1]}, b^{[3]}, \ldots, b^{[k]}$
- Train distinct learner (i.e., tree) on each $b^{[i]}$ to produce k classifiers/regression trees
- Classify new instance by classifier vote (equal weights) or a mean (regression)

Bagging algorithm

Input: Dataset $\mathcal{D}$, base learner, number of bootstraps M

1. For $m = 1, \ldots, M$ do:
   a. Draw a bootstrap sample $\mathcal{D}^{[m]}$ from $\mathcal{D}$.
   b. Train the base learner on $\mathcal{D}^{[m]}$ to obtain model $b^{[m]}(x)$.
2. Aggregate the predictions of the M estimators to determine the bagging estimator.

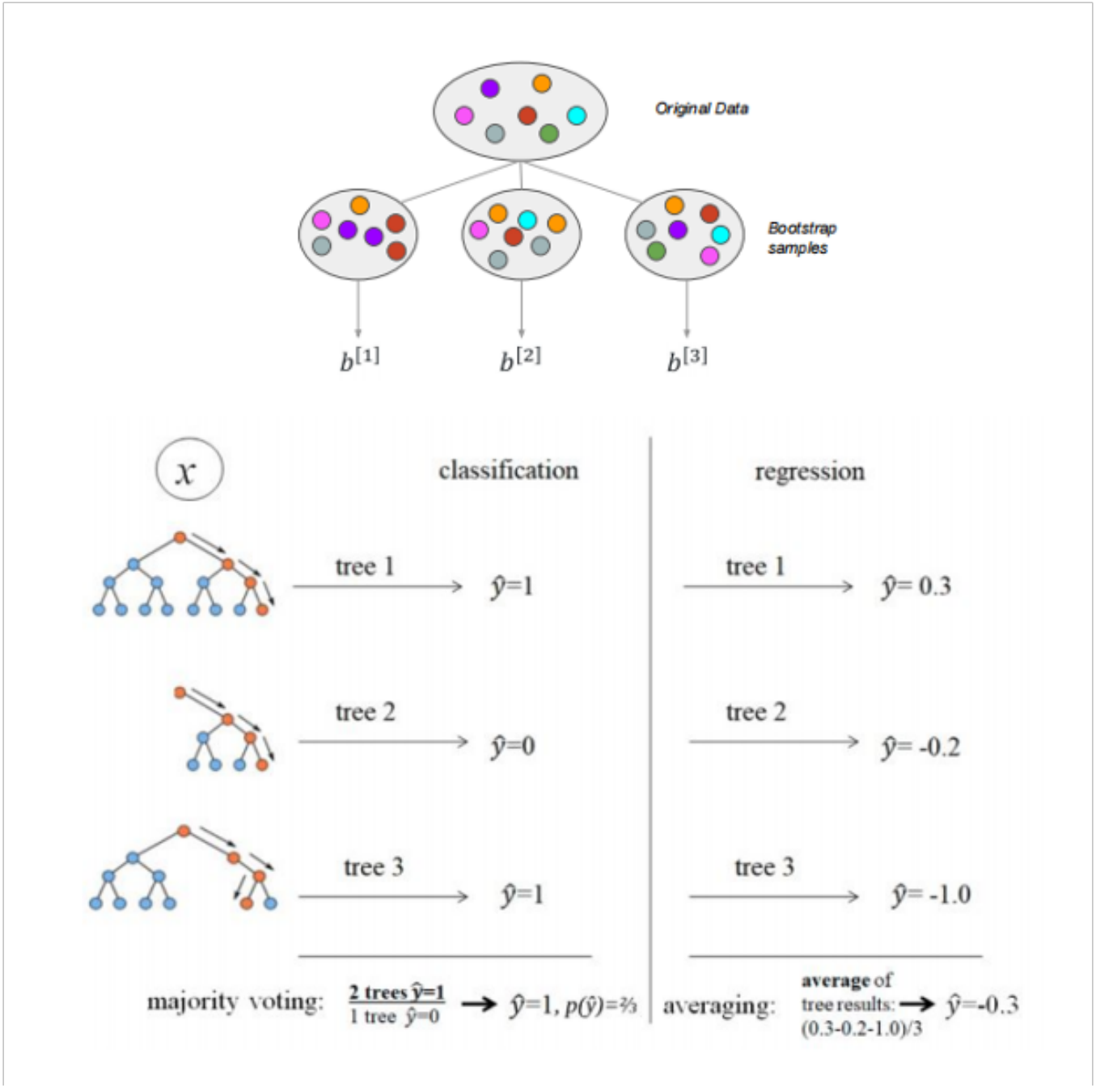
a. Regression:

$$f^{[M]}(x) = \frac{1}{M} \sum_{m=1}^{M} b^{[m]}(x)$$

b. Classification:

$$f^{[M]}(x) = \arg\max_{k \in \mathcal{Y}} \sum_{m=1}^{M} \mathbb{I}\left(b^{[m]}(x) = k\right)$$

Illustration of bagging with trees as base learners



Random forest by Breiman (2001)

Idea: Bagging with Decision Trees on Random Subset of Features

Random forest algorithm

Input: Dataset $\mathcal{D}$ of n observations, number of trees M, number of variables `mtry` to consider at each split.

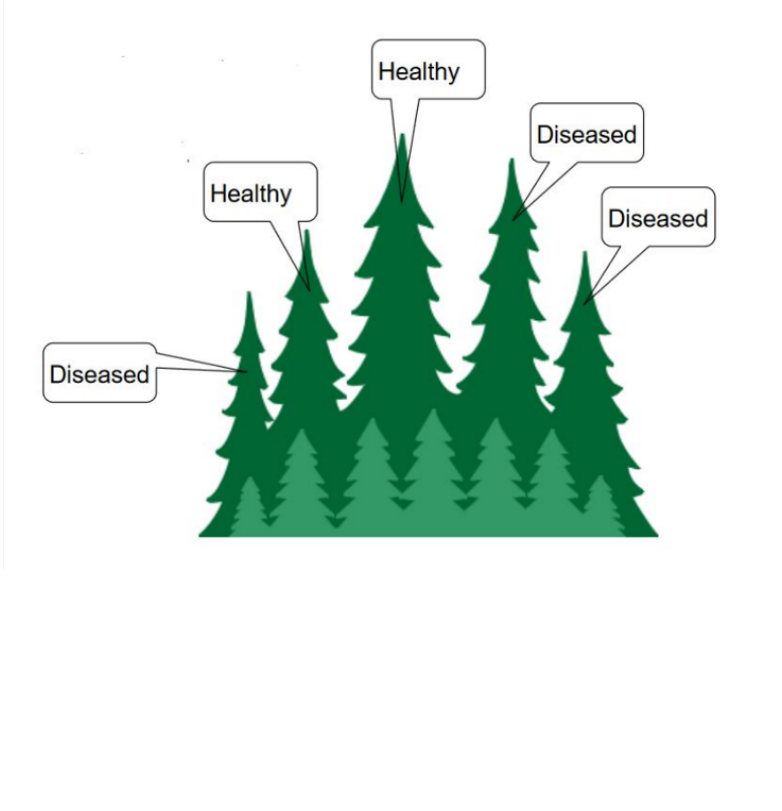
For $m = 1, \ldots, M$:

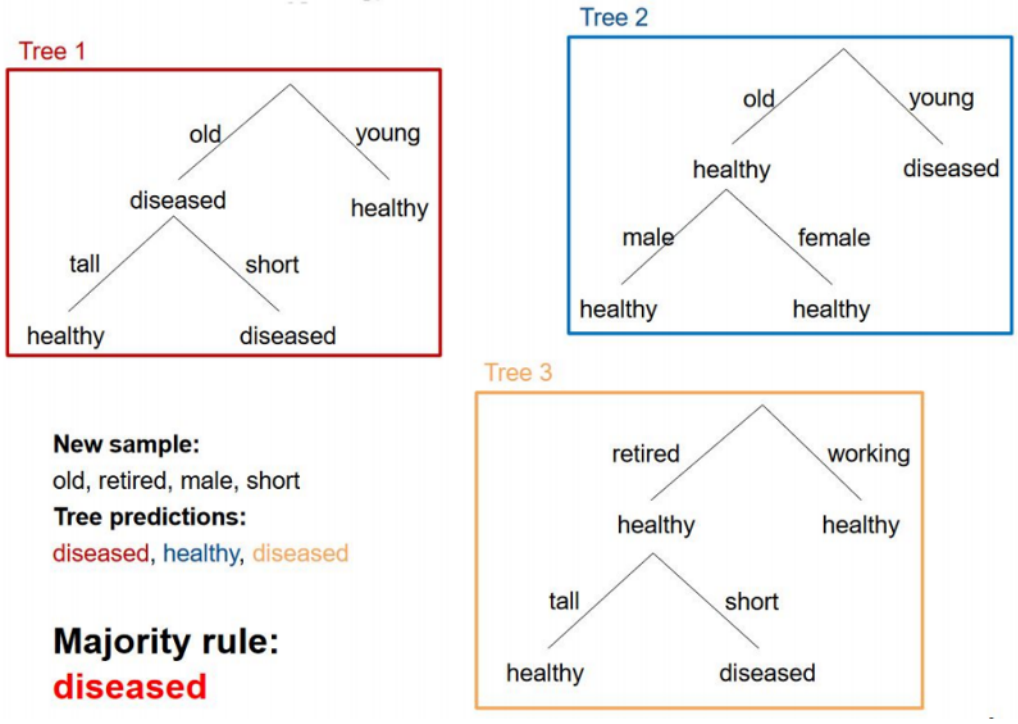
1. Draw a bootstrap sample $\mathcal{D}^{[m]}$ from $\mathcal{D}$.
2. Grow a tree $b^{[m]}(x)$ using $\mathcal{D}^{[m]}$.
3. At each split, randomly select `mtry` candidate features.
4. Choose the best split only among these `mtry` features.
5. Grow the tree fully, without early stopping or pruning.

Aggregate the predictions of the M trees to predict new data:

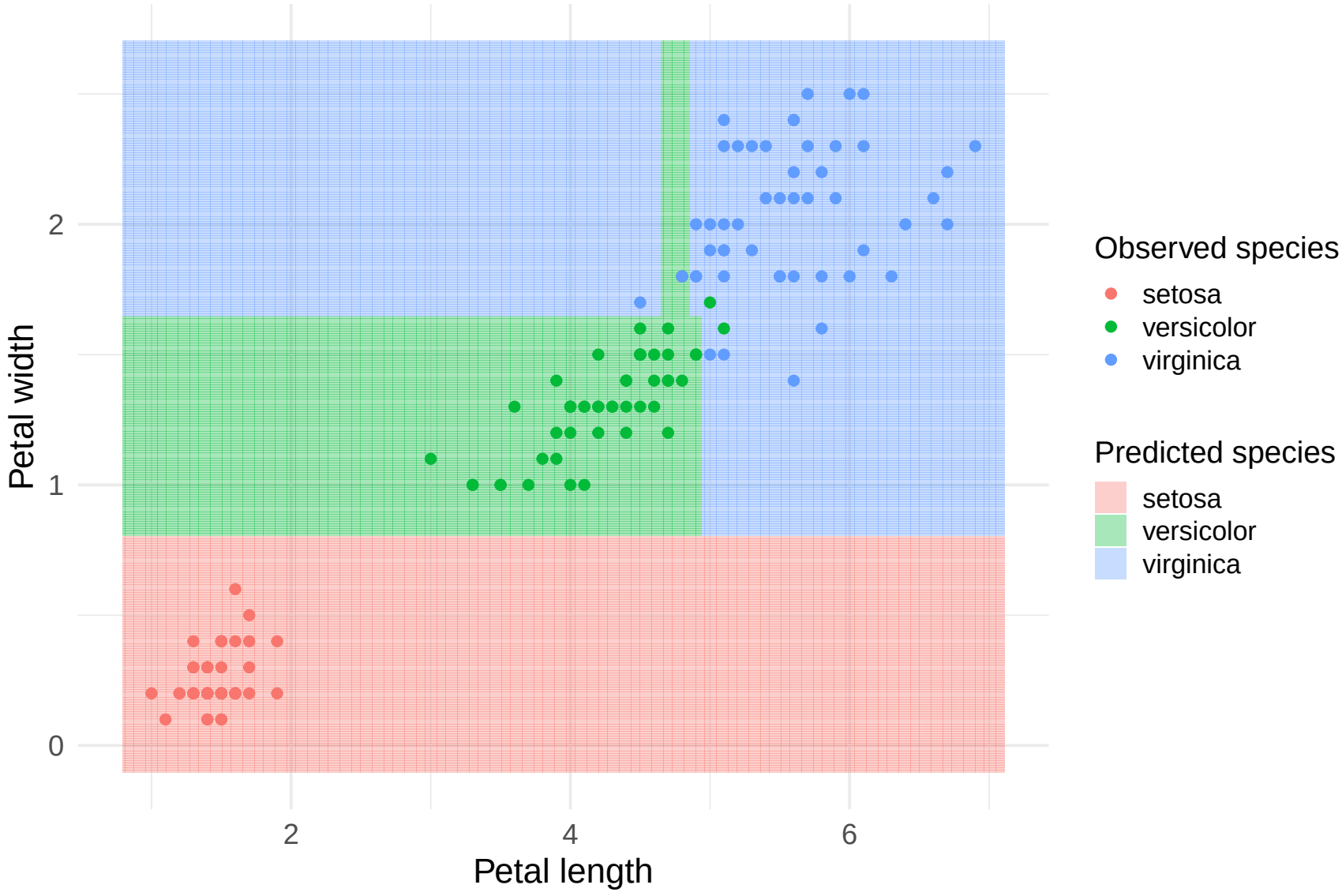
- Regression: $f^{[M]}(x) = \frac{1}{M} \sum_{m=1}^{M} b^{[m]}(x)$
- Classification: $f^{[M]}(x) = \arg\max_{k \in \mathcal{Y}} \sum_{m=1}^{M} \mathbb{I}\left(b^{[m]}(x) = k\right)$

Random forest – illustration

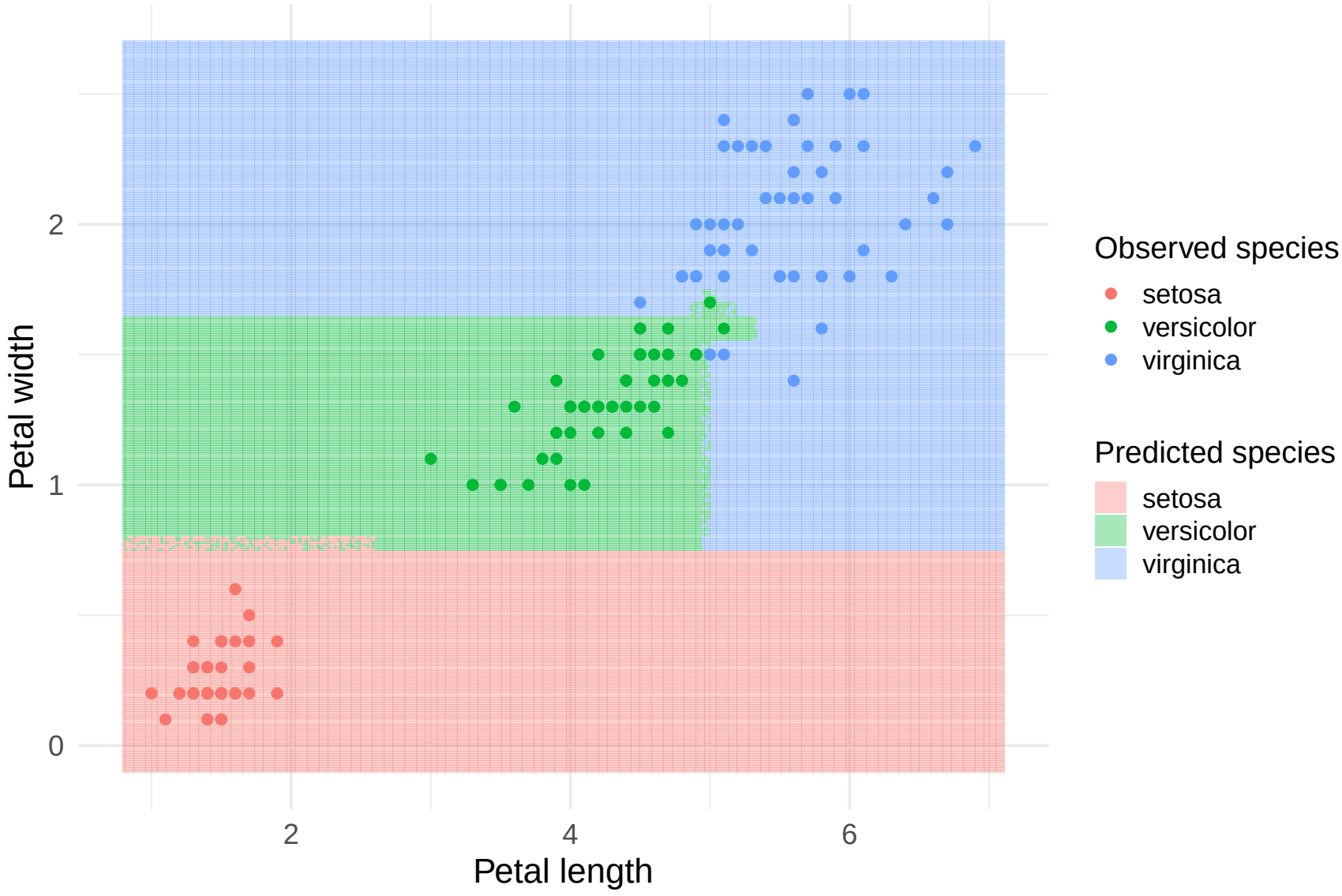




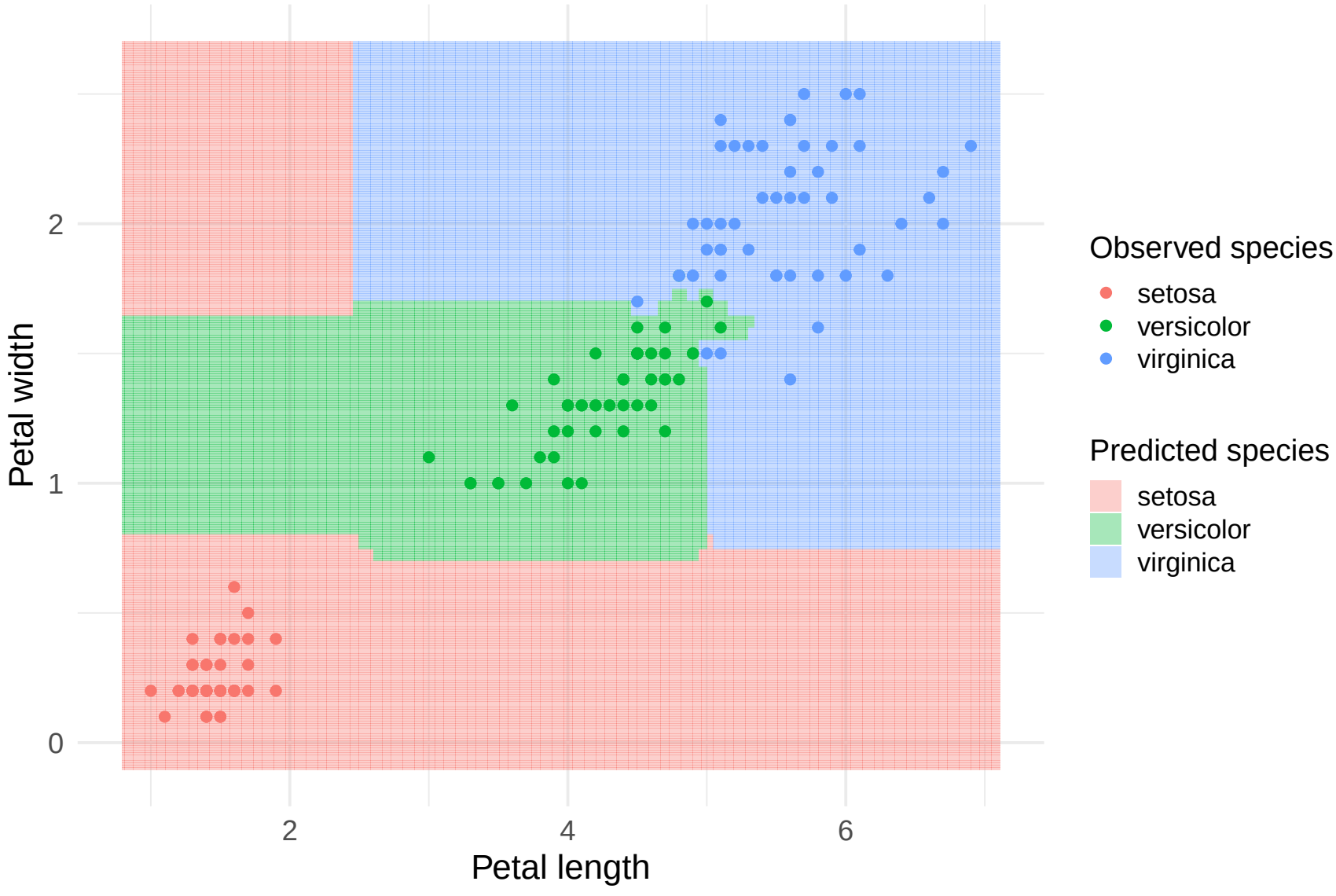
Random Forest with 1 tree



Random Forest with 10 trees



Random Forest with 500 trees



Random forest – final notes

- Random forests can improve on bagged trees through a small modification that decorrelates the trees, reducing variance when their predictions are averaged.
- We build multiple decision trees on bootstrapped training samples.
- When building these trees, each time a split is considered, a random subset of predictors is chosen as candidate split variables from the full set of p predictors.
- A fresh subset of `mtry` predictors is selected at each split. Typically, we choose `mtry` $\approx \sqrt{p}$ for classification.
- R package: `randomForest`

When does an ensemble help?

- Suppose each binary classifier is correct with probability 0.51.
- Let $X_i = 1$ if classifier i is correct, and $X_i = 0$ otherwise: $P(X_i = 1) = 0.51$
- For an ensemble of M independent classifiers, majority voting is correct with probability

$$P\left(\frac{1}{M}\sum_{i=1}^{M} X_i > 0.5\right) = \sum_{j=\lfloor M/2 \rfloor + 1}^{M} \binom{M}{j} 0.51^j (0.49)^{M-j}.$$

- For $M = 2$, strict majority requires both classifiers to be correct: $P\left(\frac{X_1+X_2}{2} > 0.5\right) = 0.51^2 = 0.2601$.
- If classifiers are highly correlated, the ensemble helps less; if they make identical errors, it is no better than one classifier.

A good ensemble: strong base learners, many learners, low correlation between learners.

Tree, forest, bagging and boosting

- Decision tree
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - XGBoost

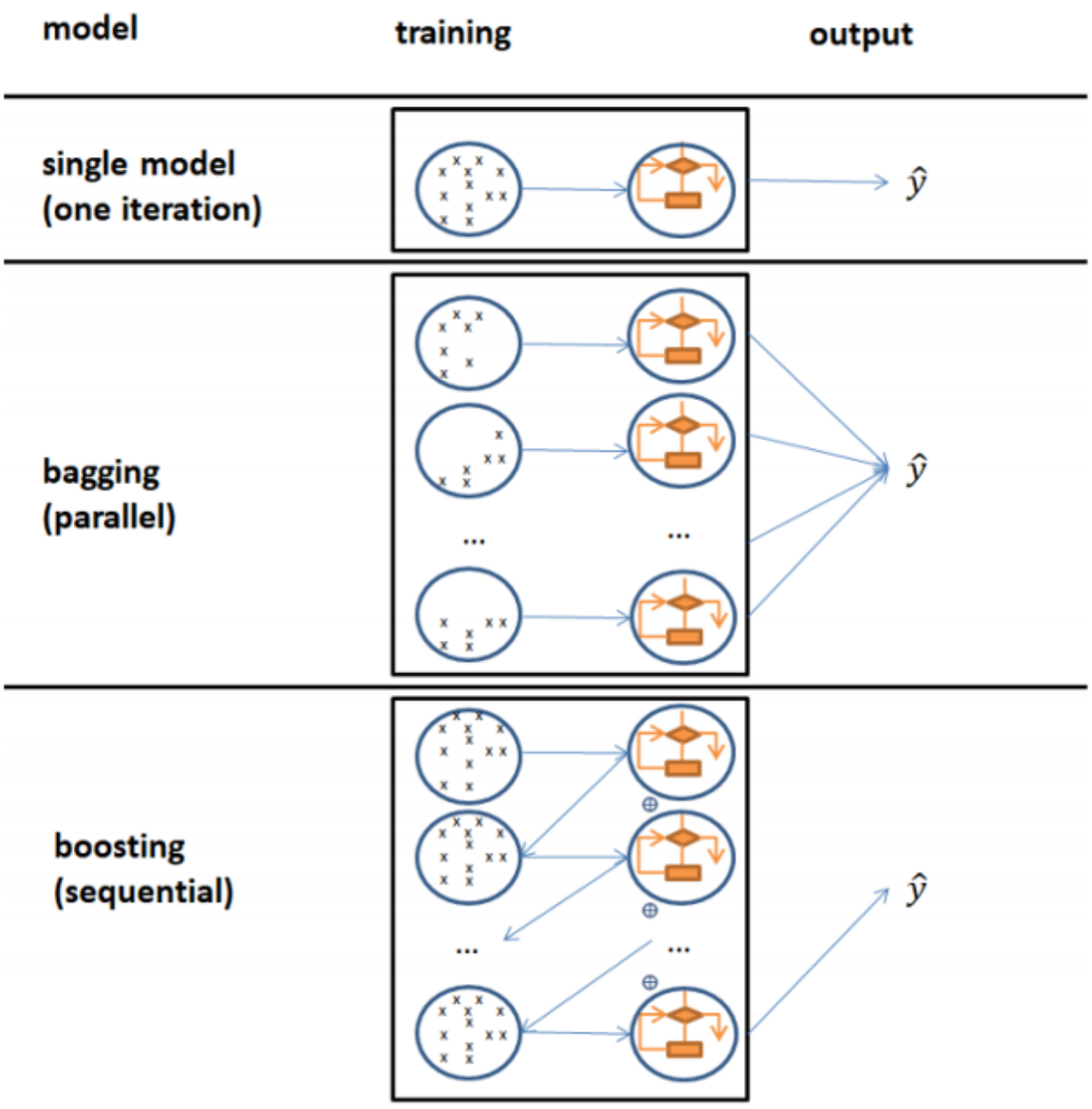
Bagging vs. boosting

- Like bagging, boosting is a general approach that can be applied to many statistical learning methods for regression or classification.
- **Bagging:** create multiple bootstrap samples, fit a separate tree to each sample, and combine the trees into one predictive model.
- In bagging, trees are built independently of one another, so they can be trained in parallel.
- **Boosting:** trees are also combined into an ensemble, but they are grown sequentially.
- Each new tree is trained to improve on the errors made by the previous trees.
- We discuss boosting using decision trees as an example, but the same idea can be generalized to other models.

Boosting – idea

- Unlike a single large decision tree, which can fit the data too aggressively and overfit, boosting learns slowly.
- Starting from the current model, we fit a small decision tree to the residuals.
- We then add this tree to the fitted function, updating the model and the residuals.
- Each tree is usually shallow, with only a few terminal nodes.
- By repeatedly fitting small trees to the residuals, boosting gradually improves $\hat{f}(x)$ in regions where the current model performs poorly.
- A shrinkage parameter λ slows the learning process further, allowing many small trees to contribute to the final model.

Boosting – illustration



Boosting for regression trees

1. **Initialize the model**

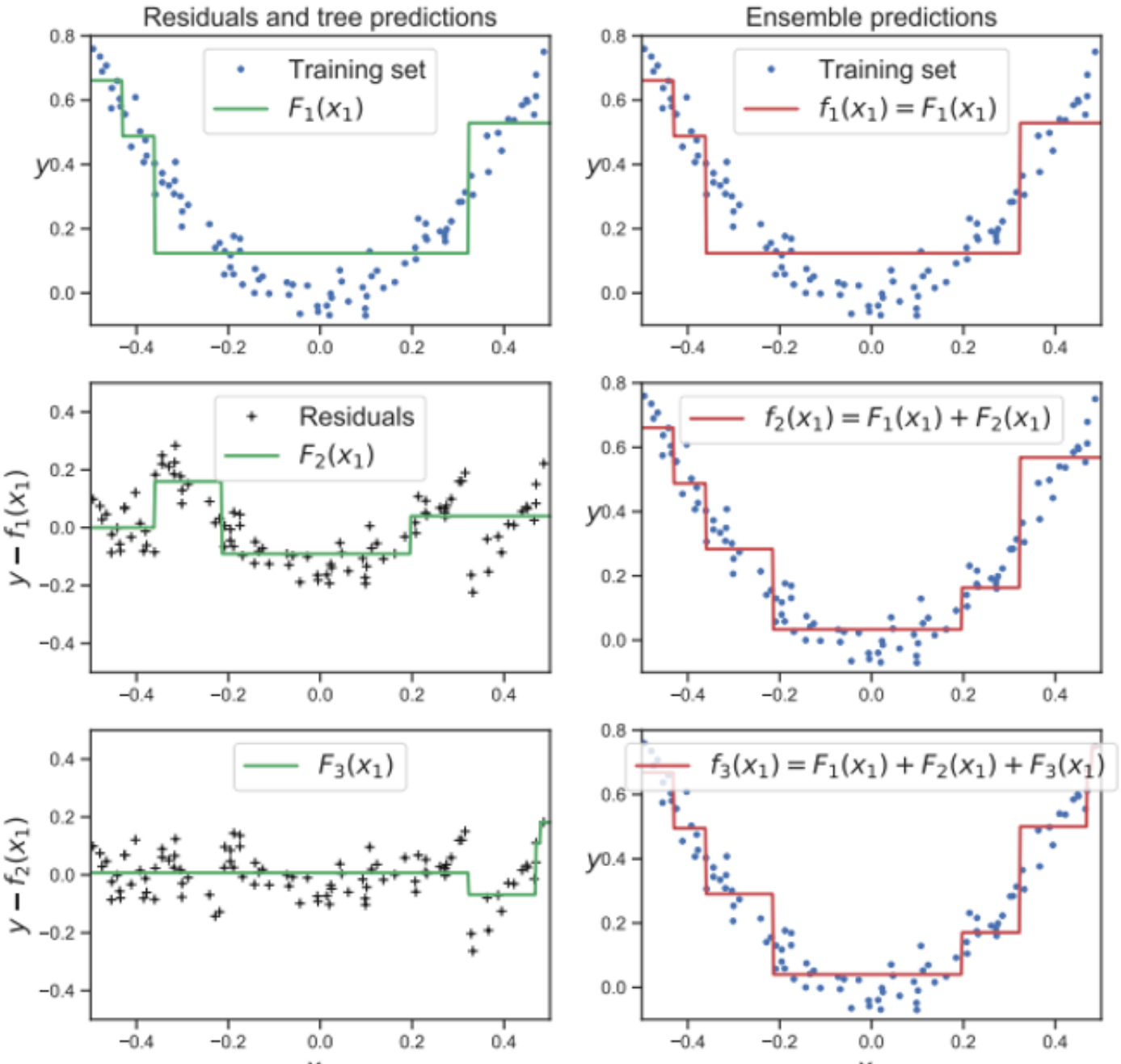
$\hat{f}(x) = 0$, $r_i = y_i$, $i = 1, \ldots, n$ where r_i is the residual for observation i, and y_i is the observed response.

2. **For** $b = 1, \ldots, B$:
   - Fit a small regression tree $\hat{f}_b$ with d splits, that is, $d + 1$ terminal nodes, to the data (X, r).
   - Here, the current residuals r_i are used as the response values.
   - Update the fitted function: $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}_b(x)$
   - Update the residuals: $r_i \leftarrow r_i - \lambda \hat{f}_b(x_i)$, $i = 1, \ldots, n$

3. **Final boosted model**

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}_b(x)$$

Boosting for regresion trees – ilustraion



Parameters to tune in boosting

- **Number of trees** B
  - Controls how many trees are added to the ensemble.
  - Too large a value can lead to overfitting, especially if λ is not small.
- **Shrinkage parameter** λ
  - A small positive number that controls how much each tree contributes.
  - Typical values are between 0.01 and 0.001.
  - Smaller λ usually requires a larger number of trees B.
- **Number of splits** d in each tree
  - Controls the complexity of each tree in the boosted ensemble.
  - If $d = 1$, each tree is a stump, meaning it has one split and two terminal nodes.
  - Stumps often work well in practice.

Bagging as an additive model

- More generally, consider an additive model

$$f(x) = \sum_{j=1}^{M} \beta_j F_j(x), \qquad L(f) = \sum_{i=1}^{N} l\{y_i, f(x_i)\},$$

where $F_j(x)$ are base learners, β_j are their weights, $L(f)$ is the empirical loss of the model f and $l\{y_i, f(x_i)\}$ is the loss for one training observation i.

- In **bagging**, the weights are usually fixed:

$$\beta_j = \frac{1}{M},$$

and each $F_j(x)$ is fitted independently, often on a bootstrap sample.

Boosting as an additive model

- In **boosting**, the model is built sequentially by adding weak learners one at a time.
- After m steps, the fitted model is

$$f_m(x) = \sum_{j=1}^{m} \beta_j F_j(x).$$

- The next learner is chosen to reduce the empirical loss:

$$(\beta_{m+1}, F_{m+1}) = \arg\min_{\beta, F} \sum_{i=1}^{N} l\left\{y_i, f_m(x_i) + \beta F(x_i)\right\}.$$

- Thus, boosting repeatedly adds a new weak learner that improves the current model.

Tree, forest, bagging and boosting

- Decision tree
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - **AdaBoosting**
  - Gradient Boosting
  - XGBoost

AdaBoost: idea

- **AdaBoost** stands for **adaptive boosting**.
- Main idea: build an ensemble of weak classifiers sequentially, giving more weight to observations that were misclassified by previous classifiers.

Other Training Methods

- Filtering: give misclassifications to next tree in cascade
- Resampling: increase probability of encountering misclassified instances
- Reweighting: increase cost of misclassifications (exponential loss)

AdaBoost: idea extended

- Let $F_j(x) \in \{-1, +1\}$ is a weak classifier. At step m, the current fitted model is $f_{m-1}(x) = \sum_{j=1}^{m-1} \alpha_j F_j(x)$.
- Let $\tilde{y}_i \in \{-1, +1\}$. We add a new classifier $F_m(x)$ with weight α_m by minimizing the exponential loss:

$$L_m(F, \alpha) = \sum_{i=1}^{N} \exp\left[-\tilde{y}_i \left\{f_{m-1}(x_i) + \alpha F(x_i)\right\}\right]$$

- This can be written as a weighted loss:

$$L_m(F, \alpha) = \sum_{i=1}^{N} w_{i,m} \exp\left[-\alpha \tilde{y}_i F(x_i)\right],$$

  where $w_{i,m} = \exp\left[-\tilde{y}_i f_{m-1}(x_i)\right]$.
- Therefore, observations that are poorly classified by the current model receive larger weights in the next step.

AdaBoost algorithm

AdaBoost.M1 for binary classification

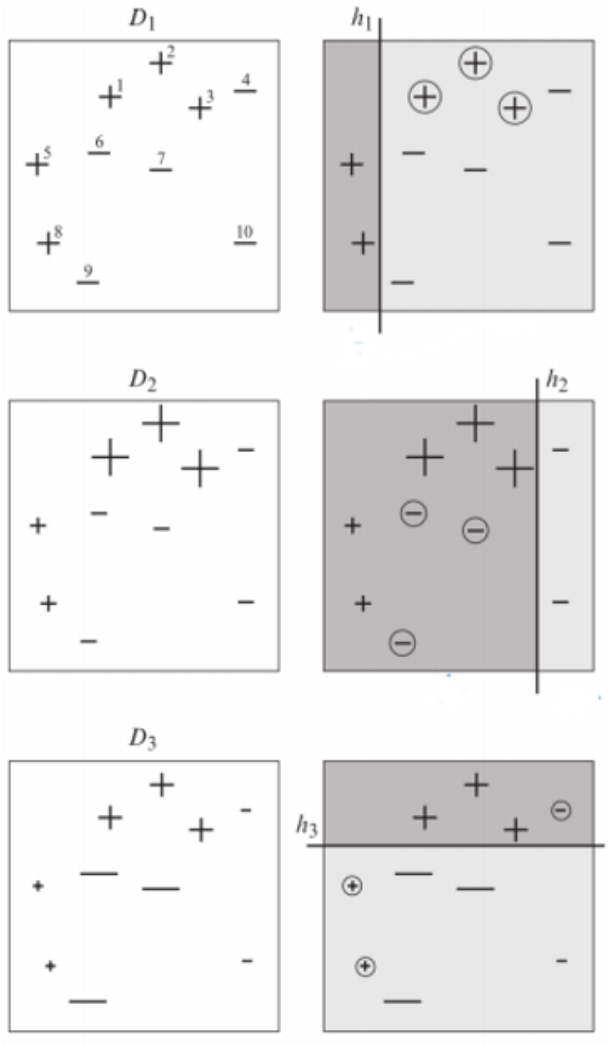
1. Initialize the observation weights: $w_{i,1} = \frac{1}{N}$, $i = 1, \ldots, N$.
2. For $m = 1, \ldots, M$:
   - Fit a weak classifier $F_m(x)$ to the training data using weights $w_{i,m}$.
   - Compute the weighted training error:$\mathrm{err}_m = \frac{\sum_{i=1}^{N} w_{i,m} I\{\tilde{y}_i \neq F_m(x_i)\}}{\sum_{i=1}^{N} w_{i,m}}$.
   - Compute the classifier weight: $\alpha_m = \log\left(\frac{1-\mathrm{err}_m}{\mathrm{err}_m}\right)$.
   - Update the observation weights: $w_{i,m+1} = w_{i,m} \exp\left[\alpha_m I\{\tilde{y}_i \neq F_m(x_i)\}\right]$
   - Optionally normalize the weights so that they sum to 1.
3. Return the final classifier:

$$\hat{y}(x) = \mathrm{sign}\left(\sum_{m=1}^{M} \alpha_m F_m(x)\right)$$

.

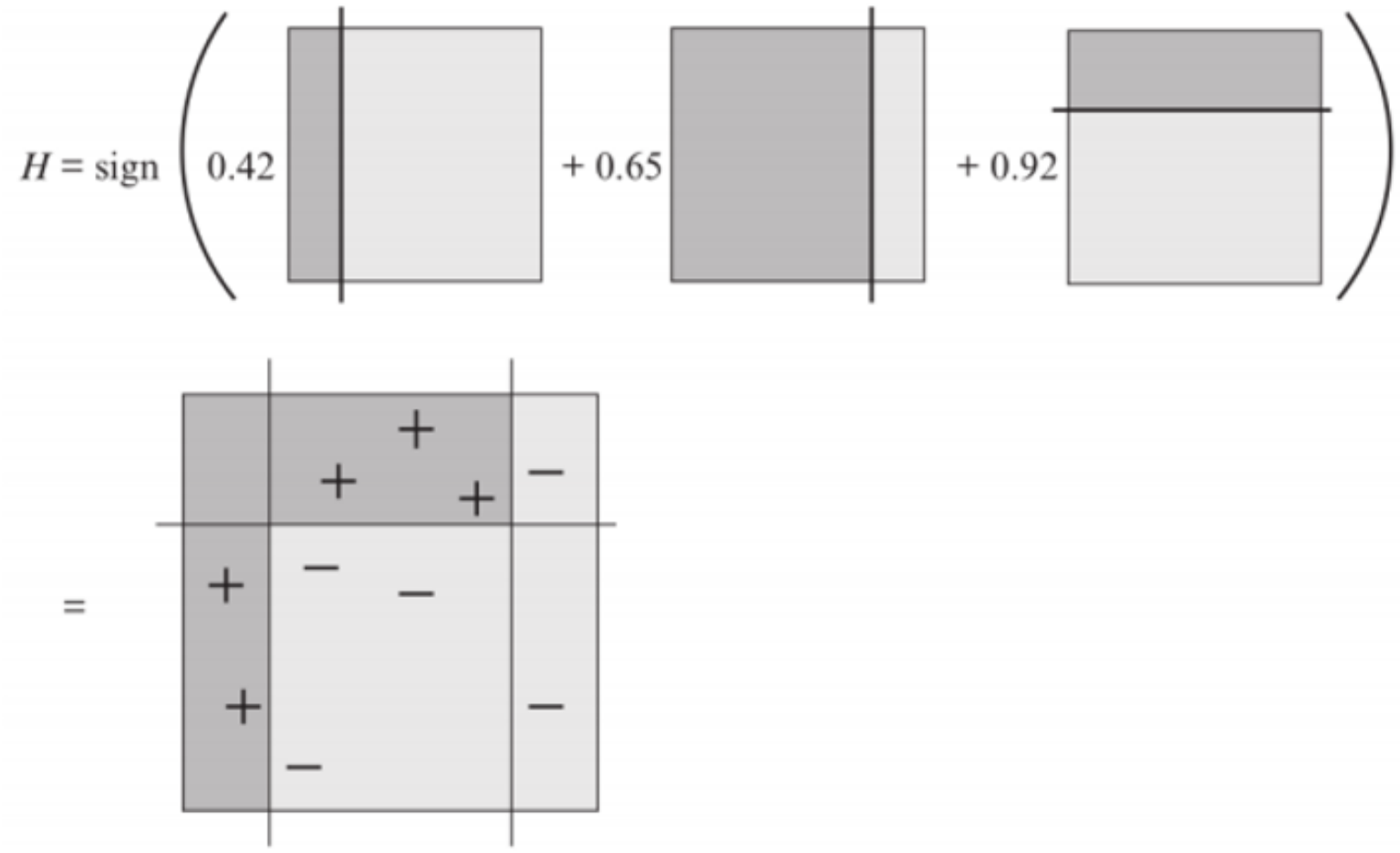
AdaBoost – example

- $n = 10$ observations and $M = 3$ iterations, using tree stumps as base learners.
- Each observation label is shown as $+$ or $-$.
- The size of each label represents the observation weight in that iteration.
- Dark grey region: the base learner at iteration m predicts $+$.
- Light grey region: the base learner at iteration m predicts $-$.



Adaboost – example

- The three base models are combined into one classifier.
- All observations are correctly classified



Tree, forest, bagging and boosting

- Decision tree
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - **Gradient Boosting**
  - XGBoost

Gradient boosting

Idea: At each step, gradient boosting fits a weak learner to the direction that most reduces the loss. These directions are given by the **negative gradients**, or **pseudo-residuals**.

At step m, let g_{im} be the **negative gradient** of the loss for observation i, evaluated at the current model f_{m-1}:

$$g_{im} = -\left[\frac{\partial \ell(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}.$$

Name	Loss	Negative gradient
Squared error	$\frac{1}{2}(y_i - f(x_i))^2$	$y_i - f(x_i)$
Absolute error	$\lvert y_i - f(x_i)\rvert$	$\text{sgn}(y_i - f(x_i))$
Exponential loss	$\exp(-\tilde{y}_i f(x_i))$	$\tilde{y}_i \exp(-\tilde{y}_i f(x_i))$
Binary log loss	$-\{y_i \log(\pi_i) + (1 - y_i)\log(1 - \pi_i)\}$	$y_i - \pi_i$
Multiclass log loss	$-\sum_c y_{ic} \log(\pi_{ic})$	$y_{ic} - \pi_{ic}$

Gradient boosting algorithm

1. **Initialize the model** $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2. **For** $m = 1, \ldots, M$:

- Compute the negative gradient, also called the pseudo-residual:

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x_i)=f_{m-1}(x_i)}, \qquad i = 1, \ldots, N.$$

- Fit a weak learner $F_m(x)$ to the pseudo-residuals:

$$F_m = \arg\min_F \sum_{i=1}^{N} \left(r_{im} - F(x_i)\right)^2.$$

- Let ν be the learning rate. Update the model as follows:

$$f_m(x) = f_{m-1}(x) + \nu F_m(x),$$

3. **Return the final model**: $f(x) = f_M(x)$.

Tree, forest, bagging and boosting

- Decision tree
  - Introduction
  - Classification and Regression Tree (CART)
- Random forest
  - Ensemble
  - Bagging
  - Random forest
- Boosting
  - AdaBoosting
  - Gradient Boosting
  - **XGBoost**

XGBoost

Tabular Data: Deep Learning is Not All You Need

Ravid Shwartz-Ziv RAVID.ZIV@INTEL.COM
IT AI Group, Intel

Amitai Armon AMITAI.ARMON@INTEL.COM
IT AI Group, Intel

Abstract

A key element of AutoML systems is setting the types of models that will be used for each type of task. For classification and regression problems with tabular data, the use of tree ensemble models (like XGBoost) is usually recommended. However, several deep learning models for tabular data have recently been proposed, claiming to outperform XGBoost for some use-cases. In this paper, we explore whether these deep models should be a recommended option for tabular data, by rigorously comparing the new deep models to XGBoost on a variety of datasets. In addition to systematically comparing their accuracy, we consider the tuning and computation they require. Our study shows that XGBoost outperforms these deep models across the datasets, including datasets used in the papers that proposed the deep models. We also demonstrate that XGBoost requires much less tuning. On the positive side, we show that an ensemble of the deep models and XGBoost performs better on these datasets than XGBoost alone.

- XGBoost is often among the best-performing methods for prediction with structured tabular data.

XGBoost

- **XGBoost** stands for **eXtreme Gradient Boosting**.
- It is a high-performance library for gradient boosting, written in C++, with interfaces for languages such as R, Python, and Julia.
- It is designed for speed and scalability, and is often faster than older gradient boosting implementations such as `gbm`.
- It has been widely used in machine learning competitions, especially for structured tabular data.
- It supports regularization, which helps control model complexity and reduce overfitting.
- It can handle missing values internally by learning default directions in the trees.

XGBoost: tree base learner and regularization

- **Base learner:** tree model $F_m(x) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(x \in R_{jm})$
- **Gradient boosting tree:**
  - Find good regions R_{jm} for tree m using standard regression-tree learning on the residuals.
  - Then optimize the terminal-node weights: $\hat{w}_{jm} = \arg\min_w \sum_{x_i \in R_{jm}} \ell\left(y_i, f_{m-1}(x_i) + w\right)$
- **Regularization:** J is the number of leaves.

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell\left(y_i, f(x_i)\right) + \Omega(f)$$

$$\Omega(f) = \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2$$

XGBoost: regularized objective

At iteration m, XGBoost adds a new tree F_m to the current model:

$$f_m(x) = f_{m-1}(x) + F_m(x).$$

The new tree is chosen by approximately minimizing

$$\mathcal{L}^{(m)} = \sum_{i=1}^{N} \ell\left(y_i, f_{m-1}(x_i) + F_m(x_i)\right) + \Omega(F_m),$$

where the tree regularization term is $\Omega(F_m) = \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2$.

Here:

- J is the number of leaves in the new tree F_m.
- w_j is the prediction value in leaf j.
- γ penalizes adding more leaves.
- λ penalizes large leaf weights.

XGBoost: regularized objective

A tree can be written as $F_m(x) = w_{q(x_i)}$, where $q(x)$ assigns observation x_i to one of the J leaves.

So the objective becomes

$$\mathcal{L}^{(m)} = \sum_{i=1}^{N} \ell\left(y_i, f_{m-1}(x_i) + w_{q(x_i)}\right) + \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2.$$

Now define the current prediction $\hat{y}_i^{(m-1)} = f_{m-1}(x_i)$.

Then we are approximating

$$\ell\left(y_i, \hat{y}_i^{(m-1)} + w_{q(x_i)}\right)$$

around the current prediction $\hat{y}_i^{(m-1)}$.

XGBoost: regularized objective

Using a second-order Taylor approximation,

$$\ell\left(y_i, \hat{y}_i^{(m-1)} + w_{q(x_i)}\right) \approx \ell\left(y_i, \hat{y}_i^{(m-1)}\right) + g_{im} w_{q(x_i)} + \frac{1}{2} h_{im} w_{q(x_i)}^2,$$

where

$$g_{im} = \left[\frac{\partial \ell(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x_i)=f_{m-1}(x_i)}, \; h_{im} = \left[\frac{\partial^2 \ell(y_i, f(x_i))}{\partial f(x_i)^2}\right]_{f(x_i)=f_{m-1}(x_i)}.$$

Substituting the Taylor approximation into the objective gives

$$\mathcal{L}^{(m)} \approx \sum_{i=1}^{N} \left[\ell\left(y_i, \hat{y}_i^{(m-1)}\right) + g_{im} w_{q(x_i)} + \frac{1}{2} h_{im} w_{q(x_i)}^2\right] + \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2.$$

XGBoost: regularized objective

The term $\sum_{i=1}^{N} \ell\left(y_i, \hat{y}_i^{(m-1)}\right)$ does not depend on the new tree F_m. Therefore, when choosing F_m, we can drop this constant term.

The approximate objective is therefore

$$\tilde{\mathcal{L}}^{(m)} = \sum_{i=1}^{N} \left[g_{im} w_{q(x_i)} + \frac{1}{2} h_{im} w_{q(x_i)}^2 \right] + \gamma J + \frac{1}{2} \lambda \sum_{j=1}^{J} w_j^2.$$

Now group observations by leaf. For leaf j, define $R_j = \{i : q(x_i) = j\}$. For all observations $i \in R_j$, $w_{q(x_i)} = w_j$.

Define the sum of first derivatives in leaf j as $G_{jm} = \sum_{i \in R_j} g_{im}$.

Define the sum of second derivatives in leaf j as $H_{jm} = \sum_{i \in R_j} h_{im}$.

XGBoost: regularized objective

Then

$$\sum_{i=1}^{N} g_{im} w_{q(x_i)} = \sum_{j=1}^{J} G_{jm} w_j, \quad \sum_{i=1}^{N} \frac{1}{2} h_{im} w_{q(x_i)}^2 = \sum_{j=1}^{J} \frac{1}{2} H_{jm} w_j^2.$$

The regularization term $\frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2$ can be combined with the second-order term:

$$\sum_{j=1}^{J} \frac{1}{2} H_{jm} w_j^2 + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2 = \sum_{j=1}^{J} \frac{1}{2}(H_{jm} + \lambda) w_j^2.$$

Therefore, the approximate objective can be written as

$$\tilde{\mathcal{L}}^{(m)}(q, w) = \sum_{j=1}^{J} \left[G_{jm} w_j + \frac{1}{2}(H_{jm} + \lambda) w_j^2 \right] + \gamma J.$$

Summary

- Decision trees are simple and interpretable models for regression and classification.
- However, individual trees are often not competitive with other methods in terms of predictive accuracy.
- Bagging, random forests, and boosting can substantially improve the predictive performance of trees.
- These methods grow many trees on the training data and combine their predictions to form an ensemble.
- Random forests and boosting are among the strongest methods for supervised learning with tabular data.
- However, their results are generally less interpretable than those of a single decision tree.

References

- ESLII: Chapter 8.7, 9, 10
- PRML Chapters 3.1.4, 3.3
- PML: Chapters 18
- Wikipedia article “decision tree learning”, “random forest” and “boosting”
- Rpackage::rpart, randomForest, boosting, XGboost